Sect 7.4: Dynamic Memory Allocation

Review

- Pass by Value
- Pass by Reference
- Pass by Pointer (aka Pass by Address)

Dynamic Memory Allocation

- new
- delete
- nullptr

Pass by Reference vs. Pass by Value: The Matrix

- bool fight(Person& neo, Person agent_smith);

Pass by Reference vs. Pass by Pointer (aka Pass by Address)

- Send the person for their vaccination.
  - void vaccine(Person& human);
- Get the address of the person. Go there to vaccinate them.
  - void vaccine(Person* humanPtr);

Ways to assign value to a pointer variable: set nullptr

- Way 1: Initialize a pointer variable to nullptr when it points to no valid memory address.
- nullptr represents a pointer that does not point to any valid memory address.
- A nullptr pointer cannot be dereferenced using the * operator.
- Dereferencing a nullptr pointer causes a **segmentation fault** (program crash).

```
int* p = nullptr;
cout << *p << endl;
// ERROR: p is nullptr — cannot be dereferenced.
// Applying * on a nullptr pointer causes
// a segmentation fault.
```

Ways to assign value to a pointer variable: use address of existing variable

- The **address** of *variableName* is obtained using &variableName.
  - The **address-of** operator & **precedes** a variable name.
- Way 2: Set a pointer variable to the **address** of a declared *variableName*.

```
int a = 5;
int* p = &a; // &a is the address of variable a.
// By int* p = &a; p stores the address of a.
// Visually, p points to a.
// *p and a refer to the same value,
// changing one changes the other.
```

Dynamic Memory Allocation

- The size of an array is not always known beforehand.
- Use **dynamic memory allocation** to request memory from the **heap** (free store) at run time.

```
int n = 5;
double* p = new double[n];
// n can be a variable
```

- Allocates an array of size n on the heap.
- p stores the address of the **first element** of the allocated array.

Ways to assign a value to a pointer: dynamically allocated memory

- Use new to dynamically allocate memory -- it returns the **address** of the allocated memory.
- Use delete to free the memory when it is no longer needed.

```
// Allocate a single double variable
double* account_pointer = new double;
// Use *account_pointer to access the allocated double

// Allocate an array of n doubles
double* account_array = new double[n]; // n can be a variable
// Use account_array as a regular array
```

Give the dynamically allocated memory back to the system when it is no longer needed

- new and delete must be used **together** to avoid **memory leak**.
- When dynamically allocated memory is no longer needed, use delete to release it.

```
delete[] account_array;
// Releases the block of memory allocated for
// the array account_array

delete account_pointer;
// Releases the single object allocated for
// account_pointer
```

Do not Use a Pointer After delete

- After calling delete, the memory is released back to the Operating System (OS) and must not be accessed again.

- The OS may immediately reassign that memory elsewhere.

Dynamic Memory Allocation – THE RULES

1. Every new must be matched by exactly one delete.
2. Use delete[] to release arrays allocated with new[].
3. After deleting, always set the pointer to nullptr.
4. Never access memory after it has been deleted (dangling pointer).
5. Violating these rules causes undefined behavior — crashes, unpredictable results, or worse.

Dynamic M - Common Errors: Table 5

Statements

`int* p;`
`*p = 5;`
`delete p;`

`int* p = new int;`
`*p = 5;`
`p = new int;`

`int* p = new int[10];`
`*p = 5;`
`delete p;`

`int* p = new int[10];`
`int* q = p;`
`q[0] = 5;`
`delete p;  delete q;`

`int n = 4;`
`int* p = &n;`
`*p = 5;  delete p;`

++ by Cay
All rights
reserved

non Errors:

Statements	Error

`int* p;`
`*p = 5;`
`delete p;`
	There is no call to `new int`.

`int* p = new int;`
`*p = 5;`
`p = new int;`
	The first allocated memory block was never deleted.

`int* p = new int[10];`
`*p = 5;`
`delete p;`
	The delete[] operator should have been used.

`int* p = new int[10];`
`int* q = p;`
`q[0] = 5;`
`delete p;  delete q;`
	The same memory block was deleted twice.

`int n = 4;`
`int* p = &n;`
`*p = 5;  delete p;`
	You can only delete memory blocks that you obtained from calling new.

Horstmann Copyright © 2008 by John Wiley & Sons. All rights reserved

Dangling Pointers

- A dangling pointer is a pointer that points to memory that has already been deleted.
- Accessing a dangling pointer is a run-time error.
- The freed memory may be reused by the OS — accessing it can cause data corruption or undefined behavior.

```
int* values = new int[n];
// Process values
…
delete[] values; //values now dangling
// Some other work …
values[0] = 42; //ERROR
```

Big C++ by Cay Horstmann Copyright © 2008 by John Wiley & Sons. All rights reserved

Avoiding Dangling Pointers

- To prevent a dangling pointer, assign nullptr to any pointer after calling delete:

```
int* values = new int[n];

//TODO: Process values…

delete[] values;
   //values now dangling

values = nullptr;
  //makes pointer safe
```

Big C++ by Cay Horstmann Copyright © 2008 by John Wiley & Sons. All rights reserved

Avoiding Dangling Pointers Analog

Memory as Real Estate:

- new (Acquisition): Apply for a plot of land. The system grants a deed and assigns a specific address (e.g., 123 Main St).
- delete (Relinquishment): When the land is no longer needed, return the deed so the plot can be reused.
- nullptr (Clearing the GPS): After returning the land, clear the address from your records. Setting it to nullptr prevents you from accidentally navigating to a location that is no longer yours.

Common Error: Memory Leaks

- A memory block that is never deallocated is called a <u>memory leak</u>.
- If you allocate a few small blocks of memory and forget to deallocate them, this is not a huge problem.
- When the program exits, all allocated memory is returned to the operating system.

Big C++ by Cay Horstmann Copyright © 2008 by John Wiley & Sons. All rights reserved

Memory Errors as Real Estate Disasters (from AI)

- **Memory Leak** *(The "Lost Deed"):* You acquire a house (new), but lose the deed (pointer). The city (OS) can't reclaim it, and you can't access or sell it. If this keeps happening, the entire town fills up — **no memory left to allocate**.
- **Dangling Pointer** *(The "Old GPS"):* You sell the house back to the city (delete), but forget to clear the address from your GPS (pointer). Driving there later means walking into a house that **belongs to someone else** — or one that no longer exists.

new and delete must use together

- Every new must have a corresponding delete.
- Missing delete causes a **memory leak**.
- Too many memory leaks may **crash** the program.

new and delete must use together： II

- **有借有还，再借不难** *"If you return what you borrow, borrowing again won't be hard."*

借 (borrow)

还 (return)

再借不难 (easy to borrow again)

new — allocates memory from heap

delete — releases memory back to heap

No memory leak — program runs smoothly